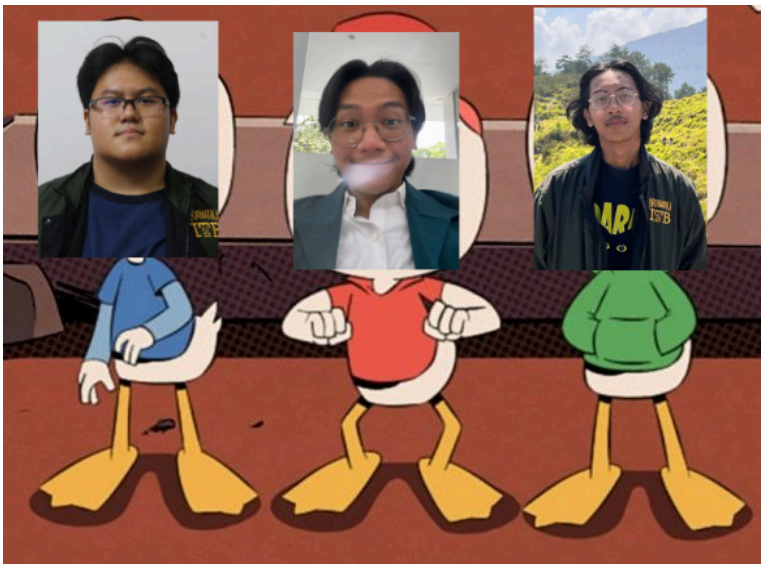
IF2211 STRATEGI ALGORITMA

LAPORAN TUGAS BESAR 2



Disusun oleh:

Kelompok 23 – AyamCarbonara

Ahmad Zaky Robbani	(13524045)
Hakam Avicena Mustain	(13524075)
Nicholas Luis Chandra	(13524105)

Dikumpulkan pada:

25 Maret 2026

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Daftar Isi 2

BAB I 4

Deskripsi Tugas 4

BAB II 5

Landasan Teori 5

2.1. Algoritma BFAlgoritma BFS 5

BAB III 6

Aplikasi BFS dan DFS 6

3.1. Proses mapping persoalan Battlecode 6

BAB IV 7

Implementasi dan Pengujian 7

4.1. Implementasi 3 alternatif solusi 7

BAB V 8

Kesimpulan dan Saran 8

5.1. Kesimpulan 8

5.2. Saran 8

Lampiran 9

Pernyataan tidak melakukan kecurangan yang ditandatangani 9

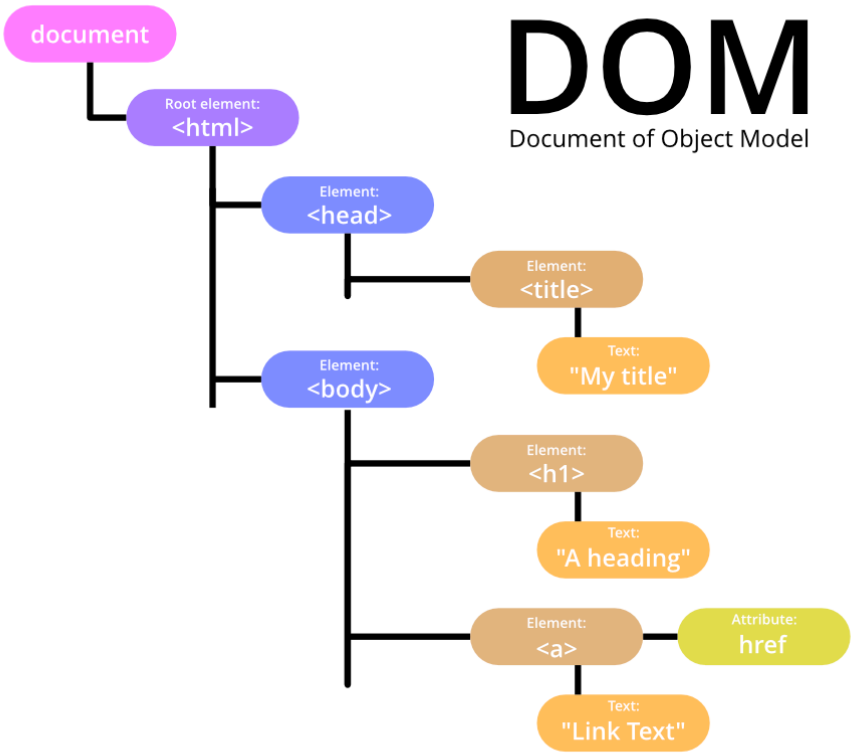
Tautan repository GitHub dan video 9

Tabel Pembagian kerja 9

Daftar Pustaka 10

BAB I

Deskripsi Tugas



Gambar 1. Struktur Pohon DOM

https://www.geeksforgeeks.org/java/what-is-document-object-in-java-dom/

Document Object Model (DOM) merupakan representasi terstruktur dari dokumen HTML dalam bentuk tree yang memungkinkan setiap elemen pada halaman web diakses, dimodifikasi, dan dimanipulasi oleh program. Struktur ini merepresentasikan hubungan antar elemen HTML seperti parent, child, dan sibling sehingga memudahkan pengolahan dokumen secara terprogram.

Struktur sebuah file HTML pada umumnya meliputi elemen root <html> yang berisi dua bagian utama yaitu <head> dan <body>. Bagian <head> berisi metadata seperti

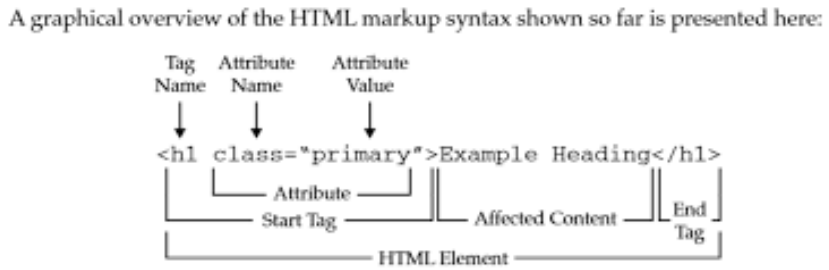
judul halaman, link stylesheet, dan script, sedangkan <body> berisi konten utama halaman seperti teks, gambar, tombol, dan elemen HTML lainnya yang ditampilkan kepada pengguna.

Sebuah website menampilkan sebuah DOM melalui sebuah file bernama Cascading Style Sheets (CSS) yang bertujuan mendeklarasikan visualisasi elemen-elemen pada pohon. Deklarasi visualisasi ditempelkan pada elemen-elemen yang berkaitan melalui CSS Selector.

Pada Tugas Besar 2 Strategi Algoritma ini, mahasiswa diminta untuk menelusuri dan mencari elemen pada struktur DOM berdasarkan CSS Selector tertentu dengan menggunakan algoritma penelusuran graf yaitu Breadth First Search (BFS) dan Depth First Search (DFS). Algoritma tersebut digunakan untuk menjelajahi struktur pohon DOM guna menemukan elemen yang sesuai dengan selector yang diberikan.

Komponen-komponen penting yang terdapat pada tugas besar ini antara lain:

1.1. Tags



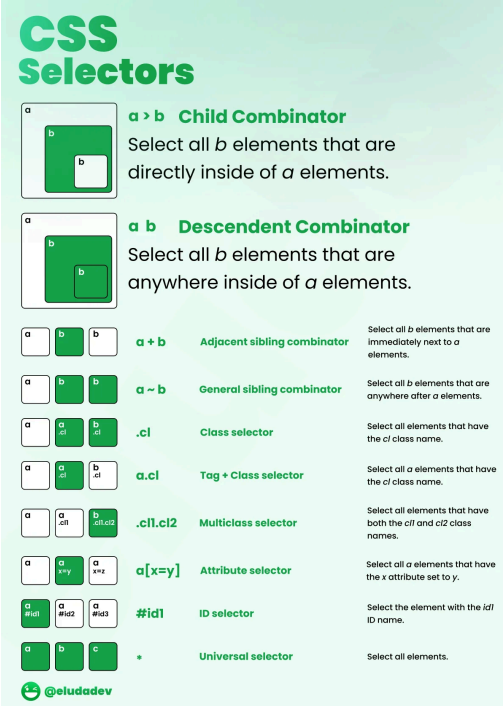
Gambar 2. Struktur Syntax HTML
https://tutorial.techaltum.com/htmlTags.html

Tag pada HTML merupakan penanda (markup) yang digunakan untuk mendefinisikan struktur dan jenis elemen dalam sebuah dokumen HTML. Tag memberi tahu browser bagaimana suatu bagian konten harus ditafsirkan dan ditampilkan pada halaman web.

Secara umum, tag HTML ditulis menggunakan tanda kurung sudut (< >). Sebagian besar elemen HTML memiliki pasangan tag pembuka dan tag penutup. Tag pembuka menandai awal suatu elemen, sedangkan tag penutup menandai akhir elemen tersebut. Tag penutup ditulis dengan menambahkan tanda garis miring / sebelum nama tag. Terdapat beberapa jenis tag HTML yang self-closing.

Dalam struktur DOM, setiap tag HTML direpresentasikan sebagai node pada pohon DOM. Hubungan antar tag membentuk struktur hierarki seperti parent, child, dan sibling . Struktur inilah yang memungkinkan dokumen HTML diproses sebagai sebuah pohon ketika dilakukan penelusuran menggunakan algoritma seperti BFS atau DFS.

1.2. CSS Selector



Gambar 3. Visualisasi CSS Selector

https://www.reddit.com/r/webdev/comments/ur6v5m/css_selectors_visually_explained

CSS Selector adalah mekanisme pada CSS untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan style. Dengan selector, kita dapat menentukan elemen mana yang akan dikenai aturan visual seperti warna, ukuran, atau tata letak. Beberapa selector dasar yang umum digunakan adalah sebagai berikut.

- Tag Selector

  Memilih elemen berdasarkan nama tag HTML. Contoh: p memilih semua elemen <p>.

- Class Selector

  Memilih elemen yang memiliki atribut class tertentu, ditulis dengan tanda titik (.).Contoh: .box

- ID Selector

  Memilih elemen dengan atribut id tertentu, ditulis dengan tanda pagar (#).

  Contoh: #header

- Universal Selector

  Memilih semua elemen HTML.

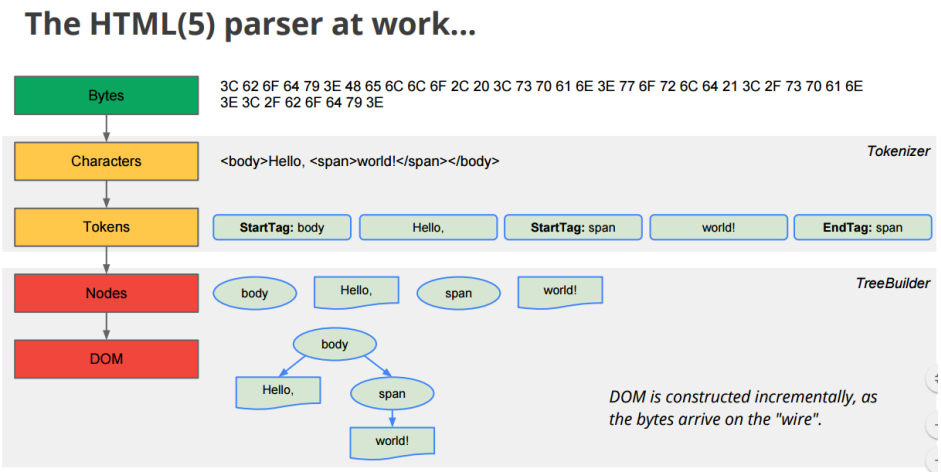
  Contoh: *

  Combinator merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari:

- Child Combinator (>)
- Descendent Combinator (“ “)
- Adjacent Sibling Combinator (+)

- General Sibling Combinator (~)

1.3. HTML Parsing



Gambar 4. Proses Parsing Pohon HTML

https://stackoverflow.com/questions/3150293/how-does-a-parser-for-example-html-work

HTML Parsing adalah proses membaca dokumen HTML dan mengubahnya menjadi struktur data pohon (tree) yang merepresentasikan hubungan antar elemen HTML. Pada proses ini, parser memproses dokumen HTML secara berurutan, mengenali tag pembuka dan tag penutup, kemudian menyusunnya ke dalam struktur hierarki.

Setiap tag HTML direpresentasikan sebagai node pada pohon. Tag yang berada di dalam tag lain akan menjadi child node, sedangkan tag yang membungkusnya menjadi parent node. Node paling atas pada struktur ini disebut root, yang umumnya merupakan elemen <html>. Dari node root tersebut, elemen-elemen lain seperti <head> dan <body> akan menjadi cabang yang membentuk struktur pohon DOM.

Hasil dari proses parsing adalah DOM Tree, yaitu representasi pohon dari dokumen HTML yang memungkinkan elemen-elemen pada halaman web ditelusuri, diakses, dan dimanipulasi secara terstruktur. Struktur pohon ini kemudian dapat digunakan dalam proses penelusuran elemen, misalnya dengan menggunakan algoritma Breadth First Search (BFS) atau Depth First Search (DFS).

BAB II

Landasan Teori

2.1. Algoritma BFS

Breadth First Search (BFS) merupakan algoritma traversal graf yang menelusuri simpul-simpul secara melebar, yaitu mengunjungi seluruh simpul pada level yang sama terlebih dahulu sebelum bergerak ke level berikutnya. Traversal dimulai dari simpul awal v, kemudian mengunjungi semua simpul yang bertetangga dengan v, lalu berpindah ke tetangga-tetangga dari simpul-simpul yang telah dikunjungi tersebut, demikian seterusnya hingga seluruh simpul terjangkau.

BFS memerlukan tiga struktur data utama. Pertama, representasi graf berupa matriks atau daftar ketetanggaan untuk merepresentasikan hubungan antar simpul. Kedua, antrian q yang bersifat FIFO (First In, First Out) untuk menyimpan simpul-simpul yang telah dikunjungi dan menunggu giliran pemrosesan tetangganya sifat FIFO inilah yang menjamin penelusuran berjalan level demi level. Ketiga, tabel boolean dikunjungi untuk menandai simpul yang sudah dikunjungi agar tidak dikunjungi dua kali.

Hasil traversal BFS dapat digambarkan sebagai pohon BFS yang mencerminkan urutan kunjungan simpul secara level per level. BFS menjamin bahwa simpul yang lebih dekat dengan simpul awal selalu dikunjungi lebih dahulu. Kompleksitas waktu BFS adalah O(N) karena setiap simpul dikunjungi tepat satu kali, sedangkan kompleksitas ruangnya adalah O(N) karena dalam kondisi terburuk seluruh simpul pada satu level tersimpan di dalam antrian secara bersamaan.

Dalam konteks penelusuran DOM Tree, BFS dimulai dari node root (<html>), kemudian mengunjungi seluruh child langsung sebelum turun ke level berikutnya. Sifat ini menjadikan BFS unggul untuk skenario pencarian elemen

yang berada dekat dengan root serta untuk mendapatkan hasil top-n elemen paling dangkal dalam hierarki DOM.

2.2. Algoritma DFS

Depth First Search (DFS) merupakan algoritma traversal graf yang menelusuri simpul-simpul secara mendalam, yaitu mendalami satu cabang sepenuhnya hingga mencapai simpul yang tidak memiliki tetangga yang belum dikunjungi, sebelum melakukan backtrack dan berpindah ke cabang berikutnya. Traversal dimulai dari simpul awal v, langsung menuju simpul tetangga pertama yang belum dikunjungi, lalu melanjutkan DFS secara rekursif dari simpul tersebut hingga cabang habis sepenuhnya.

DFS memanfaatkan call stack rekursi secara implisit sebagai mekanisme backtrack ketika semua tetangga dari suatu simpul telah dikunjungi, eksekusi kembali ke pemanggil sebelumnya secara otomatis. Tabel boolean dikunjungi tetap diperlukan untuk memastikan setiap simpul hanya dikunjungi satu kali. Hasil traversal DFS dapat digambarkan sebagai pohon DFS yang mencerminkan jalur pendalaman dan backtrack yang dilakukan. Kompleksitas waktu DFS adalah O(N), sedangkan kompleksitas ruangnya adalah O(H) di mana H adalah kedalaman pohon, karena hanya satu jalur dari root ke simpul aktif yang tersimpan di call stack pada satu waktu.

Dalam konteks penelusuran DOM Tree, DFS menyelami satu subtree elemen HTML sepenuhnya sebelum berpindah ke subtree sibling berikutnya. Sifat ini menjadikan DFS lebih efisien untuk menemukan elemen yang berada jauh di dalam hierarki DOM serta lebih hemat memori dibandingkan BFS pada pohon yang sangat lebar.

2.3. Perbandingan BFS dan DFS

Perbedaan mendasar antara BFS dan DFS terletak pada urutan penelusuran dan struktur data yang digunakan. BFS menggunakan antrian sehingga menjamin penelusuran level per level, sedangkan DFS menggunakan stack rekursi sehingga

langsung mendalami satu cabang sebelum backtrack. Tabel berikut merangkum perbandingan keduanya dalam konteks penelusuran DOM Tree:

Aspek	BFS	DFS
Struktur data utama	Antrian (Queue)	Stack
Pola penelusuran	Melebar, level per level	Mendalam, cabang per cabang
Kompleksitas waktu	O(N)	O(N)
Kompleksitas ruang	O(N) selebar tree	O(H) sedalam tree
Cocok untuk	Elemen dekat root / top-n	Elemen bersarang dalam

BAB III

Aplikasi BFS dan DFS

3.1. Aplikasi BFS

Algoritma BFS diaplikasikan untuk pencarian elemen pada pohon DOM berdasarkan CSS selector yang diberikan pengguna. Penelusuran dimulai dari node root hasil parsing HTML, kemudian seluruh node dikunjungi secara level per level menggunakan struktur antrian (queue) yang bersifat FIFO. Pada implementasi sekuensialnya, manajemen antrian dikelola secara efisien menggunakan variabel indeks head untuk meminimalisasi overhead realokasi memori saat proses dequeue, dan baru akan membebaskan memori antrian ketika indeks sudah melewati ambang batas tertentu. Pada setiap iterasi, node yang dikeluarkan dari antrian dicocokkan dengan CSS selector; jika cocok, node tersebut dimasukkan ke dalam daftar hasil, sementara seluruh child node-nya dimasukkan ke antrian untuk diproses pada iterasi berikutnya. Proses ini berlanjut hingga antrian kosong dan seluruh node pada pohon DOM telah dievaluasi.

Program juga mengimplementasikan varian BFS paralel (FastBFS) menggunakan goroutine pada bahasa Go untuk mempercepat proses penelusuran. Pada varian ini, struktur antrian dan wadah daftar hasil diubah menjadi buffered channel yang bersifat thread-safe sehingga dapat diakses secara konkuren oleh beberapa goroutine sekaligus tanpa risiko race condition. Setiap goroutine menjalankan fungsi worker yang melakukan langkah penelusuran yang sama seperti BFS sekuensial, namun dieksekusi secara paralel sehingga pemrosesan node-node pada kedalaman level yang sama dapat berlangsung secara bersamaan tanpa harus menunggu satu per satu. Pendekatan ini secara signifikan mengurangi waktu penelusuran pada dokumen HTML berukuran besar yang memiliki pelebaran node yang masif pada setiap levelnya, karena beban komputasi terdistribusi secara merata ke seluruh goroutine yang aktif.

3.2. Aplikasi DFS

Aplikasi DFS Algoritma DFS diaplikasikan untuk pencarian elemen pada pohon DOM berdasarkan CSS selector yang diberikan pengguna. Berbeda dengan BFS, penelusuran pada DFS dilakukan secara mendalam

menggunakan struktur stack yang bersifat LIFO, yaitu node yang paling terakhir dimasukkan ke stack akan diproses terlebih dahulu. Algoritma menyisipkan (push) anak elemen dari urutan paling belakang ke depan, sehingga algoritma secara otomatis memprioritaskan pendalaman satu cabang pohon DOM dari sisi kiri sepenuhnya sebelum beralih ke cabang sibling berikutnya. Pada setiap iterasi, node yang di-pop dari stack dicocokkan dengan CSS selector; jika cocok, node tersebut dimasukkan ke dalam daftar hasil, lalu seluruh child node-nya di-push ke stack untuk ditelusuri.

Program juga mengimplementasikan varian DFS paralel (FastDFS) yang memanfaatkan struktur double-ended queue (deque) berbasis sinkronisasi mutex sebagai pengganti stack konvensional. Pendekatan ini dipilih untuk mengatasi masalah ketidakseimbangan beban komputasi (work starvation) antar goroutine pada DFS stack paralel murni, sebuah thread dapat terjebak menelusuri satu cabang yang sangat dalam sementara thread lain sudah menganggur karena telah menyelesaikan cabang yang dangkal.

Dengan modifikasi deque, setiap worker memiliki deque-nya masing-masing. Worker akan beroperasi secara LIFO dengan mengambil elemen dari ujung bawah deque-nya sendiri (popBottom) sehingga perilakunya persis seperti stack DFS biasa. Namun, jika deque-nya kosong, worker tersebut akan melakukan work-stealing, yakni mencuri antrian pekerjaan dari ujung atas deque milik worker lain secara acak (stealTop). Mekanisme pengambilan hibrida LIFO secara lokal dan FIFO saat mencuri ini menjaga contention (bentrokan kunci akses data) antar thread tetap minimal, sekaligus memastikan seluruh worker tetap aktif dan produktif sepanjang penelusuran berlangsung sehingga pemanfaatan CPU berjalan maksimal.

BAB IV

Implementasi dan Pengujian

4.1. Implementasi

Algoritma BFS diimplementasikan menggunakan struktur data queue eksplisit (dengan menyimpan indeks head). Algoritma dimulai dengan memasukkan root node ke queue. Selama queue tidak kosong, lakukan

1. Keluarkan (dequeue) satu node (disebut current) dari queue.
2. Masukkan (enqueue) semua node anak dari current ke queue.
3. Jika current memenuhi predikat selektor, current dimasukkan ke daftar hasil.

Algoritma multithreading BFS memanfaatkan goroutine dari bahasa Go untuk mempercepat pemrosesan DOM. Container dari queue dan hasil diubah menjadi channel yang thread-safe. Goroutine menjalankan fungsi worker yang melakukan hal yang sama seperti algoritma BFS biasa di atas.

Algoritma DFS diimplementasikan dengan struktur data stack. Algoritma dimulai dengan memasukkan root node ke stack. Selama stack tidak kosong, lakukan

1. Keluarkan (pop) satu node (disebut current) dari stack.
2. Masukkan (push) semua node anak dari current ke stack.
3. Jika current memenuhi predikat selektor, current dimasukkan ke daftar hasil.

Algoritma multithreading DFS diimplementasikan secara tidak murni. Algoritma ini memanfaatkan struktur data double-ended queue (deque), alih-alih menggunakan stack seperti DFS biasa. Hal ini dilakukan untuk mencegah suatu thread terjebak di salah satu cabang yang dalam sedangkan thread lain menganggur karena telah menyelesaikan cabangnya yang cenderung dangkal. Setiap worker memiliki deque masing-masing. Jika dequenya tidak kosong, deque akan memproses dequenya sendiri. Jika dequenya kosong, ia akan mengambil dari deque worker lain. Pemrosesan yang dilakukan sama dengan DFS biasa.

Berikut merupakan hasil Implementasi

bfs.go

```go
func BFS(root *model.DOMNode, filter selector.Selector) []*model.DOMNode {
	queue := []*model.DOMNode{root}
	result := make([]*model.DOMNode, 0, 32)

	head := 0
	for head < len(queue) {
		current := queue[head]
		head++

		queue = append(queue, current.Children...)

		if filter(current) {
			result = append(result, current)
		}

		if head > 1024 {
			copy(queue, queue[head:])
			queue = queue[:len(queue)-head]
			head = 0
		}
	}

	return result
}
```

dfs.go

```go
func DFS(root *model.DOMNode, filter selector.Selector) []*model.DOMNode {
	stack := []*model.DOMNode{root}
	result := make([]*model.DOMNode, 0, 32)

	for len(stack) > 0 {
		current := stack[len(stack)-1]
		stack = stack[:len(stack)-1]

		if filter(current) {
			result = append(result, current)
		}

		for i := len(current.Children) - 1; i ≥ 0; i-- {
			stack = append(stack, current.Children[i])
		}
	}

	return result
}
```

lca.go

```
func FindLCA(root *model.DOMNode, id1, id2 int) *model.DOMNode {
	search1 := FastBFS(root, func(node *model.DOMNode) bool {return
node.Id == id1})
	if (search1 == nil) {
		return  nil
	}
	search2 := FastBFS(root, func(node *model.DOMNode) bool {return
node.Id == id2})
	if (search2 == nil) {
		return  nil
	}
	return FindLCAByNode(search1[0], search2[0])
}
```

4.2. Pengujian

Testcase 1

Input
<!DOCTYPE html> <html> <head>   <title>Test Complex DOM</title> </head> <body>   <header class="main-header">   <h1 id="title">Website Title</h1>   <nav>   <ul>   <li class="menu-item">Home</li>   <li class="menu-item">About</li>   <li class="menu-item special">Contact</li>   </ul>   </nav>   </header>   <main>   <section id="section1">   <article class="post">

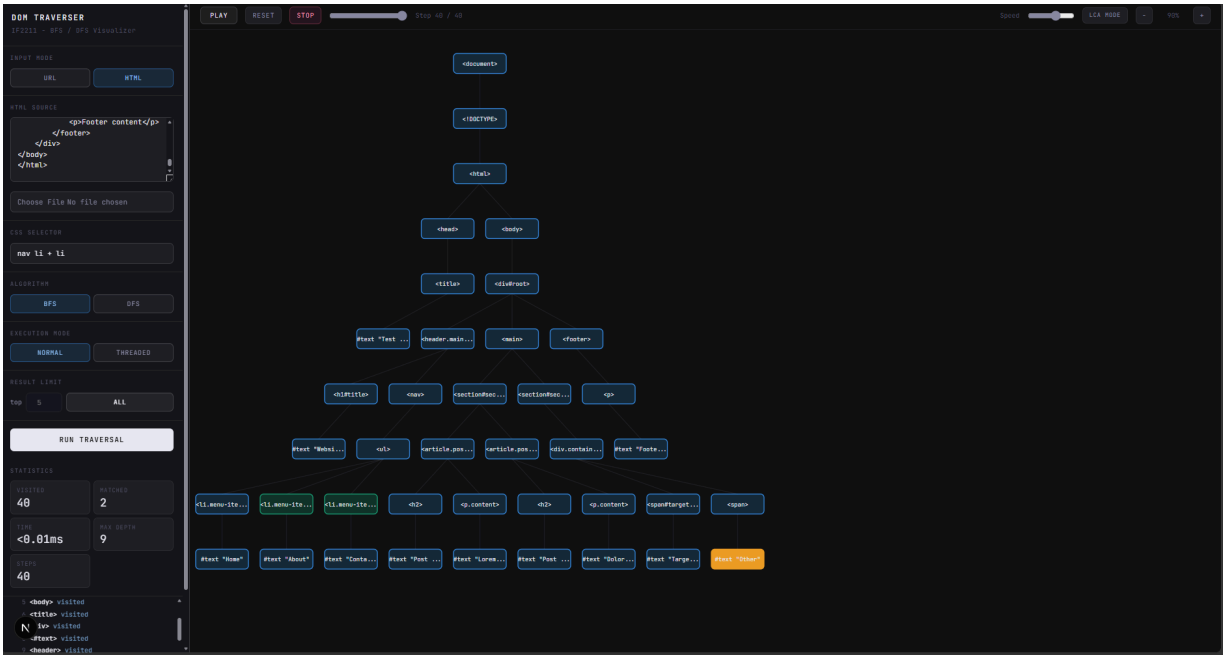
```
            <h2>Post 1</h2>
            <p class="content">Lorem ipsum</p>
          </article>
          <article class="post highlight">
            <h2>Post 2</h2>
            <p class="content">Dolor sit amet</p>
          </article>
        </section>

        <section id="section2">
          <div class="container">
            <span id="targetA">Target A</span>
            <span>Other</span>
          </div>
        </section>
      </main>

      <footer>
        <p>Footer content</p>
      </footer>
    </div>
</body>
</html>
```

CSS Selector : nav li + li

Output



Testcase 2

Input

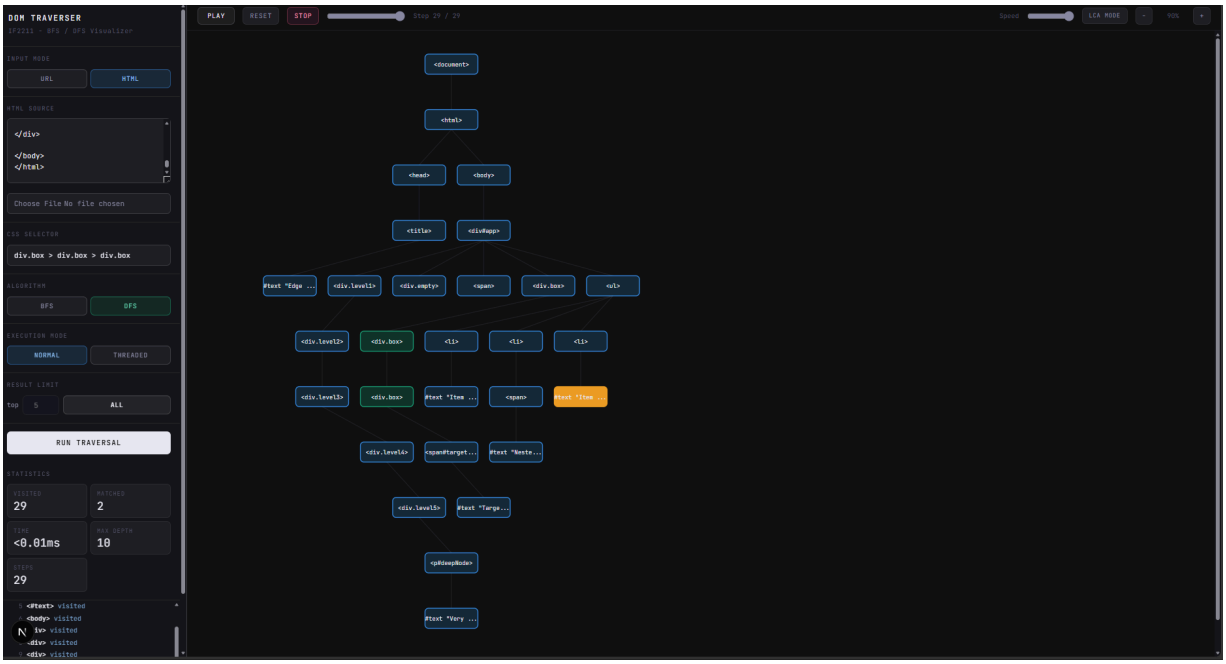
<!DOCTYPE html>
<html>
<head>
  <title>Edge Case DOM</title>
</head>
<body>
 	
  <!-- Deep nesting -->
 	
  Very Deep Node
 	
  <!-- Empty nodes -->
 	
  <!-- Duplicate classes -->
 	
  Target B
 	
  <!-- Mixed content -->
  <ul>
  <li>Item 1</li>
  <li>
  Nested Item
  </li>
  <li>Item 3</li>
  </ul>

```
</div>

</body>
</html>
```

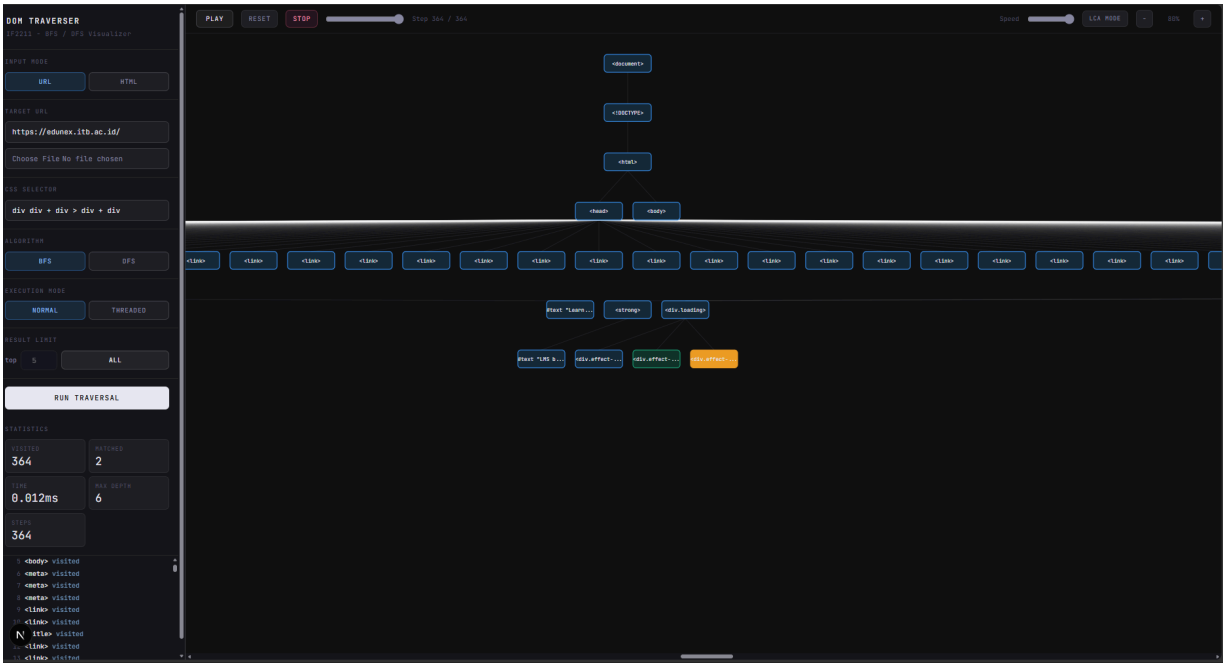
CSS Selector : div.box > div.box > div.box

Output

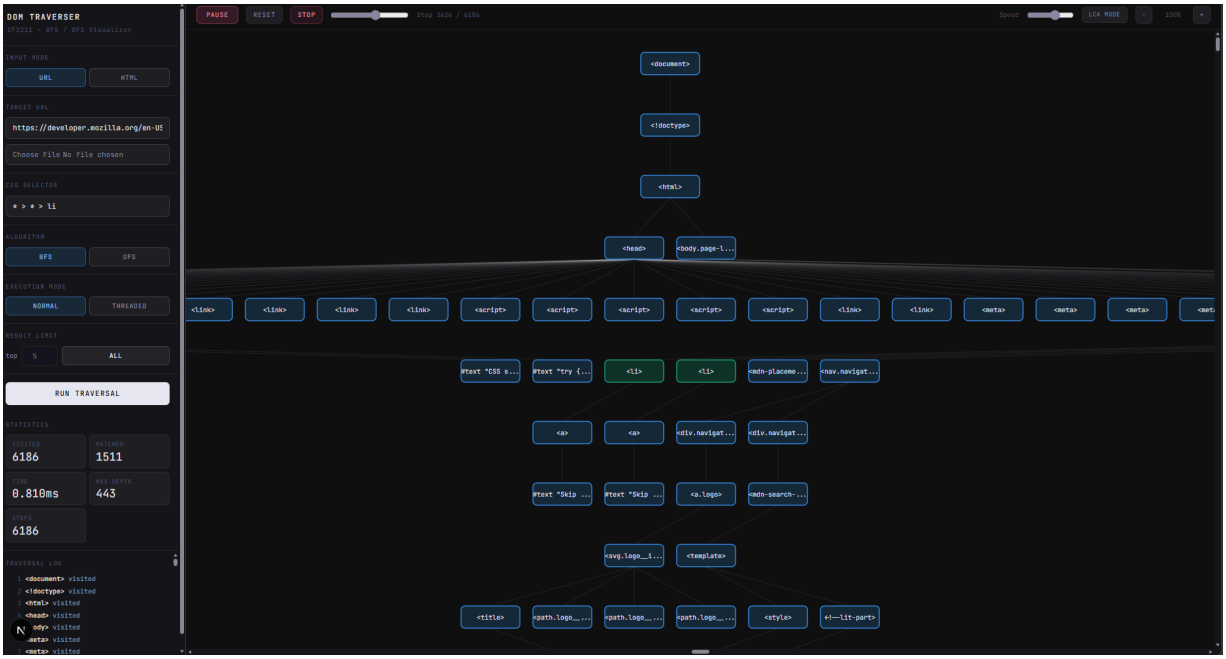


Testcase 3

Input
https://edunex.itb.ac.id/ CSS Selector : div div + div > div + div
Output



Testcase 4

Input
https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors CSS Selector : * > * > li
Output


BAB V
Kesimpulan dan Saran

5.1. Kesimpulan

Algoritma BFS dan DFS telah berhasil diimplementasikan untuk menelusuri DOM sesuai dengan strategi algoritmanya masing-masing. Tugas ini bermanfaat untuk melakukan eksplorasi mengenai web scraping dan karakteristik mengenai algoritma BFS dan DFS. Selain itu, karena pada tugas ini diimplementasikan *multithreading*, kami menemukan bahwa *multithreading* kurang direkomendasikan karena menambah kompleksitas dan untuk mayoritas kasus lebih tidak efisien karena overhead yang berlebih.

5.2. Saran

Untuk implementasi selanjutnya bisa melakukan eksplorasi lebih dalam mengenai optimasi algoritma dan penanganan kasus yang lebih menyeluruh, serta lengkap untuk CSS Selector lainnya.

Lampiran

Pernyataan tidak melakukan kecurangan yang ditandatangani

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Hakam Avicena Mustain

Ahmad Zaky Robbani

Nicholas Luis Chandra

Tautan repository GitHub dan video

Tautan GitHub: https://github.com/staplesmaster/Tubes2_AyamCarbonara

Tautan Video YouTube: https://youtu.be/CKKIQ1ueKbc?si=Ye6whDtGytje4a_R

Tabel Pembagian kerja

Tugas	Implementer
Scraping web	13524075
Parsing html	13524075
Parsing CSS selector	13524045
Algoritma BFS & DFS	13524045
API	13524105
Frontend	13524105
Laporan	13524045, 13524075, 13524105

Tabel Poin yang Dikerjakan

	Poin	Ya	Tidak
	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
	Aplikasi berhasil dijalankan	✓	
	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	

	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
	[Bonus] Membuat video	✓	
	[Bonus] Deploy aplikasi		✓
	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
	[Bonus] Implementasi multithreading	✓	
	[Bonus] Implementasi LCA Binary Lifting	✓	

Daftar Pustaka

- **CSS Selector Reference**

  https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors

- **Go Documentation**

  Documentation - The Go Programming Language

- **LCA Binary Lifting**

  Lowest Common Ancestor - Binary Lifting - Algorithms for Competitive Programming

- **Landasan Teori BFS dan DFS**

  Munir, R., & Maulidevi, N. U. (2026). *Breadth/Depth First Search (BFS/DFS) (Bagian 1)* [Bahan Kuliah IF2211 Strategi Algoritmik]. Program Studi Teknik Informatika, Institut Teknologi Bandung.

  Munir, R., & Maulidevi, N. U. (2026). *Breadth/Depth First Search (BFS/DFS) (Bagian 2)* [Bahan Kuliah IF2211 Strategi Algoritmik]. Program Studi Teknik Informatika, Institut Teknologi Bandung.